

Project Documentation

1. Project Idea / Motivation

Our intention with this project was to develop an urban garden capable of empowering communities to grow their own food in urban and densely inhabited areas. This would put new technology within people's reach, as our project enables an easier way to monitor and manage crops making use of new AI features and a set of sensors and actuators that helps to take care of plants. Running entirely on solar energy makes it sustainable and wireless, with a battery that can last for up to 1 days. Within the things the prototype is capable of performing we find: automatic watering, ventilation, detecting visual anomalies in the plants through the camera, and easy communication via telegram. There, the user can check the general status of the garden, see some photos of the plants and receive alerts.

2. Final Project Description

The prototype is built on a wooden frame, which we built ourselves. It has a pit for soil raised above the ground and also a black tarp protecting the plants from direct sunlight in the hours where it is harsher. The solar energy system is formed by a PV module, a solar power manager and batteries to ensure reliability. The camera is aimed at the plants and there are different kinds of temperature/humidity sensors. One is placed near the soil in order to analyse the microclimate close to the plants. The other one is placed above the pit to be able to measure the environmental conditions. We also have soil moisture sensors that provide information about the ground. All the sensors connect to the board which is stored inside an hermetic box at the side of the pit for protection. The wires come across the external part of the box, protected inside a waterproof coating.

The code that runs the system is split between the microcontroller, which gives the main processor access to every sensor and actuator, and the main processor, which runs the full monitoring cycle: reading the sensors, taking and analysing a photo of the plants, deciding what action is needed, storing the result, and acting on it automatically. A Telegram bot runs alongside this cycle at all times, so the user can check on the garden or receive an alert without waiting for anything (the bot is @UGardenbot; sending /start is all it takes to subscribe and start using it). Section 4 below walks through exactly how each part of the code does this.

3. Construction Process

3.1 Physical building

The building of our prototype starts with the construction of the wooden planting box. The box is 60*80cm with a depth of 30cm, it is coated with anti-humidity varnish and a plastic protection. It also has a drain to avoid water excess. In the photos we can see both the box and the structure in which it stands. all made of spruce wood sheets that we cut and assembled:



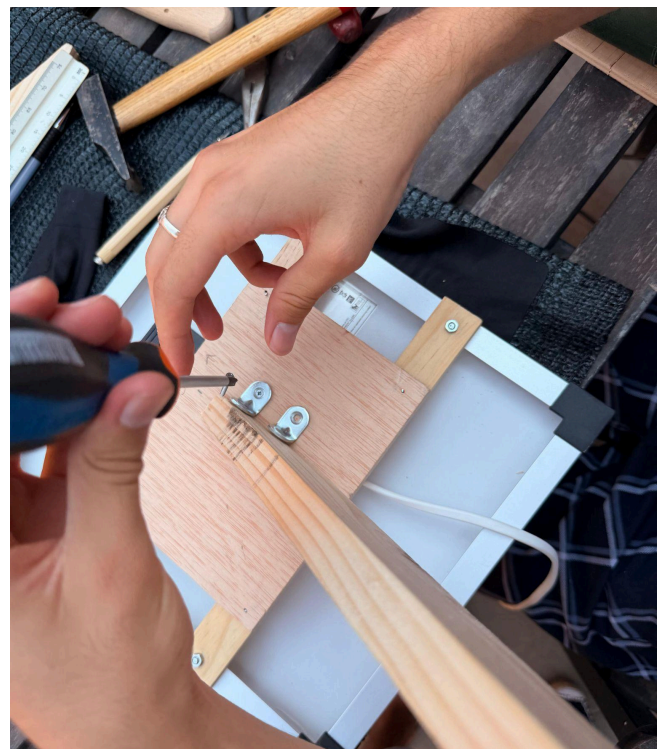


Once we had the wooden planting box, it was time to make the structure of the solar panel, the shade net, the camera and sensors and the hermetic box shelf. Both were made with spruce wood slats.





we placed the shade net and the solar panel

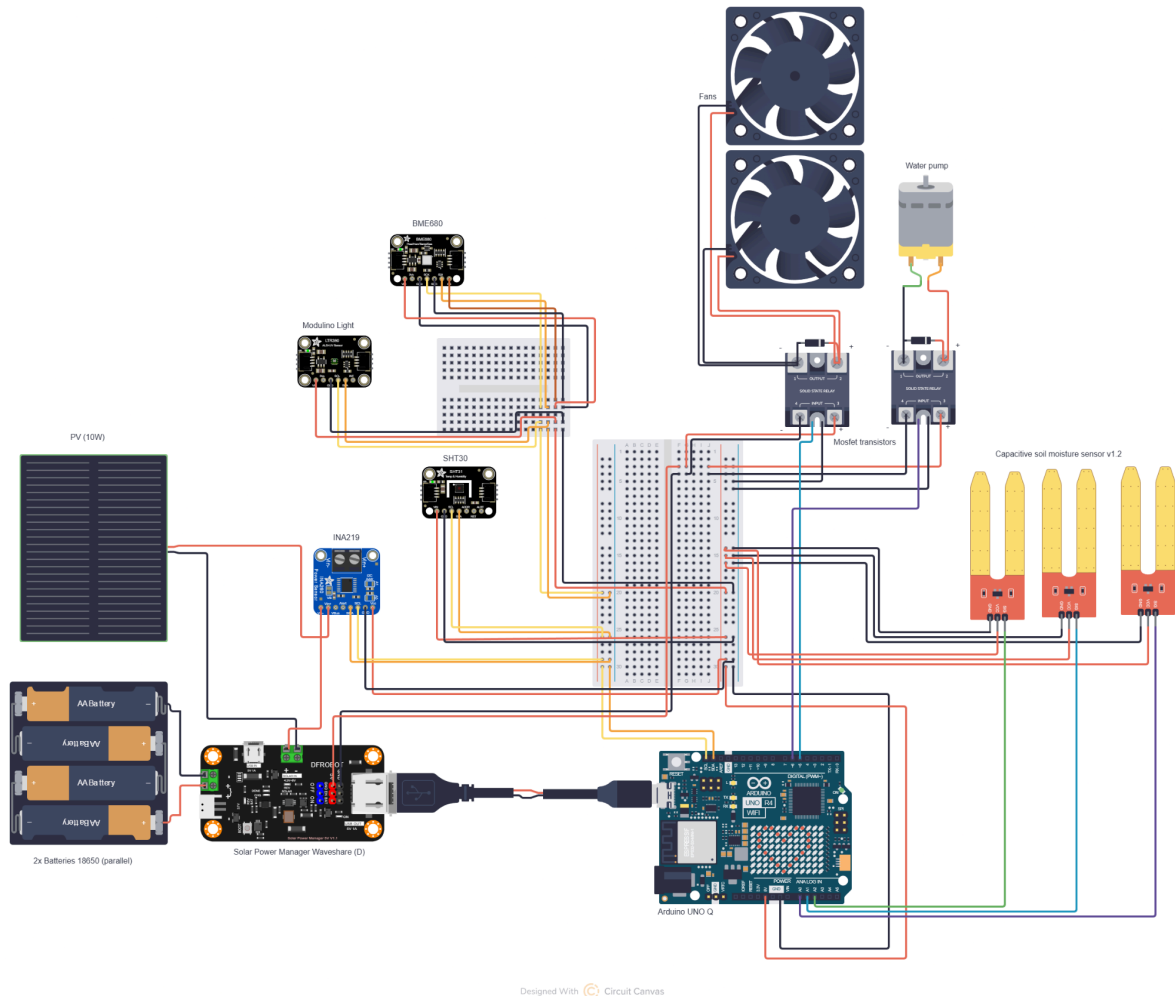




3.2 Electrical System

3.2.1 Circuit System:

Scheme:



note: the icons shown in this scheme are placeholders, not all of them correspond to real hardware, real hardware is specified below.

3.2.2 Hardware:

Main Board: Arduino UNO Q.

Power Management: Waveshare Solar Power Manager Module (D), Input: 6V-24V, Output: 5V/3A.

Power Case: sockets for up to 3 18650 batteries in parallel.

Solar Panel: Enjoy solar Monocrystalline Photovoltaic Panel, 10W, 12V.

Batteries: 2x LiitoKala 18650 Rechargeable Li-Ion, 3.7V, 3500mAh.

Actuators (Cooling): 2x JZK Brushless DC Cooling Fans, 80x80x10mm, 5V, 0.3A, 2500 rpm.

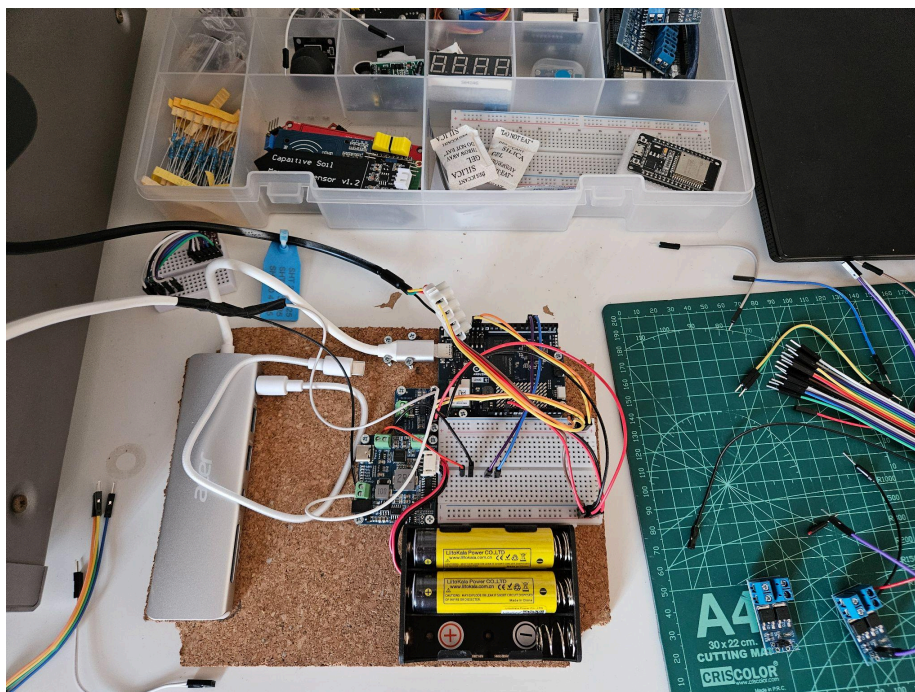
Actuators (Irrigation): 1x RUNCCI-YUN Micro Submersible Water Pumps, DC 3V-5V.

Switches: 2x Dual MOSFET Trigger Switch Driver Modules, DC 5V-36V, 400W.

USB-Hub: ACER 7-IN-1 USB C Hub

Sensors:

- **Modulino Light sensor.**
- **SHT30** Temperature and Humidity Sensor Probe, I2C output.
- **BME680** Temperature, Humidity, Pressure, VOC Sensor.
- **3x Capacitive Soil Moisture Sensors V1.2.**
- **INA219** bidirectional current and voltage sensor.



early versions of the circuit, many things left to implement.

3.2.3 Energy calculations:

POWER CONSUMPTION ANALYSIS

- BATTERY CAPACITY

Battery type: 2x LiitoKala 18650 in parallel

Nominal capacity: 7000 mAh (7 Ah)

Nominal voltage: 3.7 V

Total available energy: 25.9 Wh (7 Ah x 3.7 V)

- INDIVIDUAL COMPONENT CONSUMPTION (AT 5V)

Arduino UNO Q (Lower Active Idle): 1.0 W (200 mA) (estimated, real unknown)

Arduino UNO Q (Processing FOMO, Random Forest, camera, sensors): 3.0 W (600 mA)(estimated, real unknown)

1x Water Pump (RUNCCI-YUN): 1.0 W (200 mA)

1x Cooling Fan (JZK 8010): 1.5 W (300 mA)

PEAK BURST CONSUMPTION (WORST-CASE SCENARIO)

Note: We are fully aware that the decision-making system will never trigger the automatic irrigation and the airflow system at the exact same time. However, this calculation forces simultaneous activation to establish a strict safety margin for our hardware.

Total Burst Power: Arduino Processing (3.0 W) + 1x Pump (1.0 W) + 2x Fans (3.0 W) = 7.0 W (1400 mA)

AVERAGE POWER CALCULATION (20-MINUTE CYCLE)

Assuming the system operates in 20-minute cycles consisting of 1 minute of Peak Burst (taking decisions based on data collected by sensors and the camera) and 20 minutes of Active Idle:

Energy used during Idle (20 mins): $1.0 \text{ W} \times (20 / 60 \text{ hours}) = 0.333 \text{ Wh}$

Energy used during Burst (1 min): $7.0 \text{ W} \times (1 / 60 \text{ hours}) = 0.116 \text{ Wh}$

Total energy per 20-minute cycle: $0.333 \text{ Wh} + 0.116 \text{ Wh} = 0.449 \text{ Wh}$

Cycles per hour: $60 \text{ minutes} / 20 \text{ minutes} = 3 \text{ cycles}$

Average continuous power draw: $0.449 \text{ Wh} \times 3 \text{ cycles} = 1.347 \text{ W}$

To determine the total runtime of the prototype powered by solar energy during the night (0W solar input), the total available energy from the fully charged batteries is divided by the average continuous power draw of the system. Calculating 25.9 Wh divided by 1.347 W results in 19.23 hours. A fully charged battery will keep the system running for 20 hours under these specific conditions. So we have decided not to implement any power saving mode, as our system is theoretically able to manage it, as it won't be activating the pump and the fans in every cycle.

3.3 Final Integration + Plants

In this stage of the project, we merged the electrical system with the physical part and the plants. We also upgraded some parts of the structure in order to protect the plants and the camera from direct sunlight and ensure the prototype stability. For the plants we choose a catalan variant of tomatoes named "òptima" due to its reliability and standard behaviour.



After viewing the FOV of our camera, and the high density plants, we decide to go from 6 to 4, so there is more space in between, and the camera manages to include all the plants in its frame. In addition, we had to redo the irrigation system due to an unforeseen failure in the water pumps that forced us to replace them and redesign how the land was irrigated. This led us to the final version on 13/IX/2026:



4. Code

4.1 Sketch (MCU code)

4.1.1 Actuator.ino

String actuator_initialization() - This function prepares the digital pins connected to the water pump (pin 6) and the fans (pin 5) to be able to send signals as output. It sets the initial state of both pins to 0 or LOW.

Returns “ok” at the end of the function.

String set_actuator(int pin, int state) - Allows to change the state of a given pin, 0 = LOW, 255 = HIGH.

Returns “ok” at the end of the function.

4.1.2 bme680.ino

String bme680_initialization() - Tries up to ten times to initialize the bme680 sensor to the 0x76 I2C direction. If the initialization does succeed sets up the sensor’s oversamplings for temperature, humidity and pressure (which dictate the precision of the samples) as well as setting the filter size to 3 (which reduces noise in the samples and smoothes out the readings) Finally it turns off the air quality part of the sensor as it will not be used.

Returns “ok” if the initialization was completed, “error” otherwise.

String read_bme680() - Performs a reading and returns the result concatenated in a string, if the reading does not succeed, returns “error”.

4.1.3 ina219.ino

String ina219_initialization() - Initializes the sensor, at its default I2C address (0x40), and sets up the voltage and current configuration. Returns “ok” if the initialization is successful, “error” otherwise.

String read_ina219() - Reads voltage and current, calculates power and returns all the data concatenated into a single string.

4.1.4 modulino_light.ino

String modulino_light_initialization() - Initializes a communication with the sensor in the 0x53 direction, write(0x00) and write(0x06) initialize the sensor itself. Returns “ok” if the initialization was successful (endTransmission() = 0) “error” otherwise.

String read_modulino_light() - Fetches the infrared and ambient light measurements from the sensor. Write(0x0A) requests the infrared data, then the program reads 3 bytes and applies a bitmask that keeps only the 20 least significant bits of that 24-bit reading, which are the valid ones. Then write(0x0D) requests the ambient light reading from the sensor and the same byte treatment is performed. Returns a string with the values read concatenated.

4.1.5 moisture_v1.2.ino

String read_capacitive_moisture_sensor_v1_2(int pin_index) - Performs several readings to the given pin_index (in this case 10) and calculates the mean value of the readings between 3 and 1020 (the values outside this interval are not considered) Returns a string with the calculated mean.

4.1.6 sht_30.ino

String read_sht30() - Activates the communication with the sensor at its I2C address (which is stored as a constant value) and then asks for a reading. The reading comes in a 6 byte list where the bytes 0 and 1 correspond to the temperature and the bytes 3 and 4 are the humidity (1 and 4 are the least value bytes). Then separates the reading into two variables and applies the numerical transformation required to transform the data given by the sensor to celsius and humidity percentage. Returns a string with these two values concatenated.

4.1.7 sketch.ino

void setup() - Provides all the previous functions to the microprocessor via Bridge and then calls all the initialization functions of the sensors.

4.2 Python (MPU Code)

4.2.1 main.py

GardenState - The garden state class is the one responsible for holding all the data coming from the sensors, before all this data is stored in the csv and the Google Sheet. In fact those two methods are, for the most part, only used for storage as the vast majority of the readings of the sensor data are made via this class. It has a thread lock so there cannot be more than one access at a time to avoid data corruption.

Director - This class holds the main loop of the app, it holds a reference to each of the classes that manage each part of the project, called “orchestras”. The director’s class is decorated with the brick decorator from arduino app utils. This decorator enables the class to run simultaneously to the main flux program in a dedicated thread. This decorator also allows this class to have a start method, which is activated at the beginning of the execution and a loop method, which is executed constantly and is the core of our program.

The main loop executes the following actions:

1. Gets the data readings from the sensors
2. Executes the anomaly detection model
3. Executes the decision making model
4. Stores the data in the csv file

5. Stores the data in the Google Sheet
6. Executes an action depending on the decision model response

If any of these six steps fails, the error is logged and the cycle is skipped, instead of stopping the whole monitoring loop — a failed sensor reading or a camera error in one cycle does not take down the rest of the system.

In production, this whole process runs as a systemd service (Restart=always) on the board itself — it starts automatically when the board powers on and recovers on its own from a crash, without anyone needing to log in and start it by hand.

4.2.2 sensors.py

This file holds the classes responsible for calling the sketch functions of each sensor, transforming the strings that they return into real data and storing this data in the GardenState object.

Sensor - Parent class that holds a general structure, every type of sensor has an implementation of this class in order to receive the sensor readings and extract the data from the strings that the sketch functions return.

MoistOrchest - As there are more than one soil moisture sensor, this class has the job of gathering all the readings of the sensors, discarding the nan readings and computing the average of all the valid data (if none of the readings are valid, it also returns nan)

SensorOrchestra - Main class of the file, holds a reference to every sensor and to the GardenState object, each time its function run() is called, gathers all the sensor readings and updates the corresponding dictionary in the GardenState class.

4.2.3 vision.py

ImageOrchestra - Class responsible for handling the camera and executing the image anomaly detection model. It is separated between several functions but the class does take a picture, save it, and send it to the anomaly detection model. Once the model has analyzed the picture it returns a list with all the cells whose anomaly rating is over a certain threshold. Then it modifies the image to highlight all of those cells and stores the new image as well, finally it stores some data in the GardenState, such as the maximum anomaly score, the mean anomaly score, the list containing all the anomaly cells and the path of the output image.

4.2.4 decision.py

DecisionOrchestra - This is the class responsible for the communication with the decision making model, it gathers all the data collected in the previous steps in the GardenState objects and creates a dictionary which will be passed to the model. Then the model returns a list with every label that it keeps track of with a probability attached to it, but only the three most probable ones will be stored.

It is also important that the moisture readings are not treated as absolute values, they are rather treated relative to the previous readings as the sensors are easily affected by external factors and the absolute values of their readings are too.

4.2.5 data_manager.py

DataOrchestra - This class is responsible for storing the data gathered by the sensors and the one generated by the AI models both online, in a Google Sheet, and locally, in a csv file. The Google Sheet communication is achieved via AppScript, and the class also provides functionality that allows it to read multiple rows of the csv file, which is useful when providing moisture readings to the decision making model, as as stated before, that data is treated relative to all the previous readings.

4.2.6 actuators.py

ActuatorOrchestra - Given the data provided by the decision making model, this class holds a path of action for each of the labels tracked by the model. So, after the model has analyzed the data, this class executes the action related to the most probable label. For the most part it consists just of giving information to the user, but it is also capable of watering the plants and turning on the fans if necessary.

4.2.7 telegram.py

This file contains the TelegramDirector class, whose purpose is to wait until a user begins a conversation with the bot and sends any command. This class is the only one to be decorated with the brick decorator except the director's class. This is done in order to make this class to run in parallel in a dedicated thread so it can be listening to income commands without breaking the main flow of the code. TelegramDirector's job consists of sending messages to the user when required, and sending pictures and information when requested.

Note: apart from all those files, all the program parameters are stored in the config file for easy access throughout the program.

5. AI models

5.1 Decision making

The model we are using to decide what action the garden needs is a random forest classifier. Every cycle it receives the current sensor readings together with the anomaly score coming from the vision model, and returns the three most likely labels out of the ones it was trained on (OK, WATER, VENTILATE, ANOMALY_DETECTED, EXCESS_WATER, HYDRIC_STRESS_ALERT, and a few others), each with its own probability attached. Soil moisture is never given to the model as a raw number on its own; it is compared against a rolling baseline built from the last readings, since the absolute value of these sensors drifts over time and is not reliable by itself.

The model has never been trained on real data collected from the garden. Labelling a real history of sensor readings by hand — deciding, for each past moment, what the correct action would have been — would have taken weeks of a running garden and someone reviewing logs by hand, which did not fit in the time we had. Instead, we wrote a rule-based labeler built

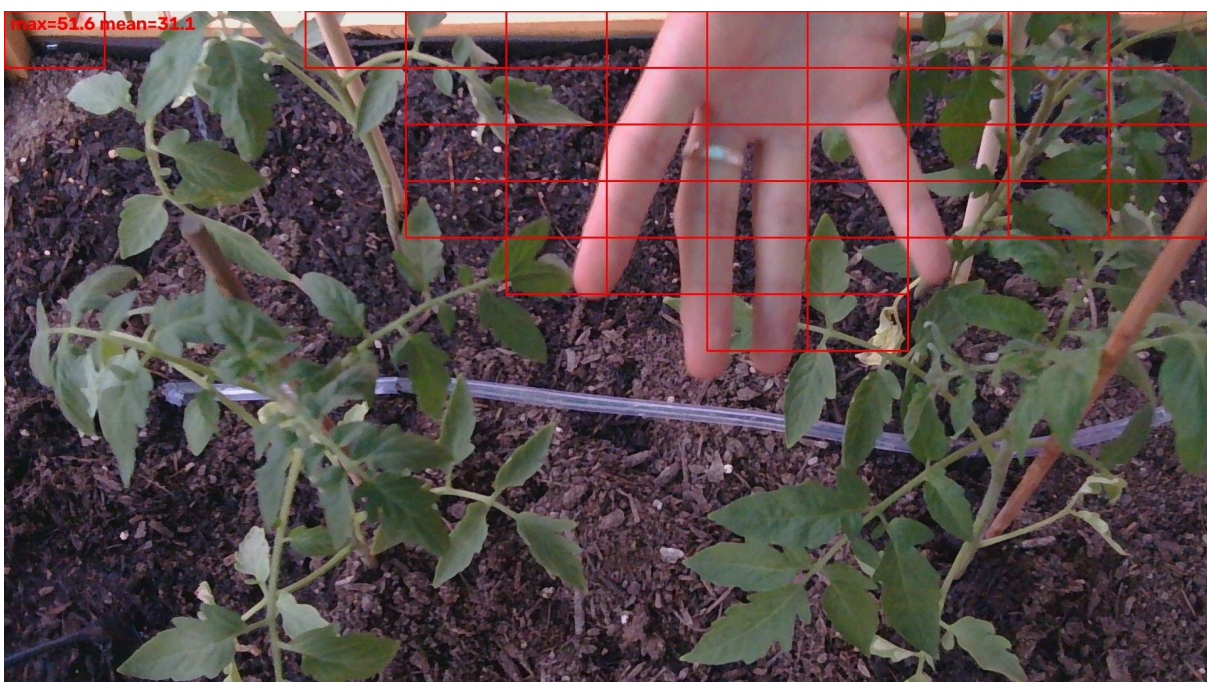
on the same thresholds the rest of the app already uses, and used it to generate thousands of plausible sensor readings together with their "correct" label. The random forest model was then trained on this synthetic dataset, learning to reproduce and generalise that rule-based logic rather than any pattern taken from a real garden.

One design problem we ran into: since the model only ever outputs one top label per cycle, two real problems happening at the same time (for example, a visual anomaly and excess water at once) compete for that same slot. We solved this by placing the anomaly check first in the labeler's rule order, so a visual anomaly always takes priority when it fires — but this means an urgent watering need could stay hidden if it happens to coincide with a camera anomaly. This is still open for a future version.

5.2 Anomaly detection

Our anomaly detection model is based on the FOMO-AD blueprint that comes in edge impulse, and has been trained with photos taken by our camera throughout one day to ensure that every kind of lightsource possible was taken into account. Given all the training photos the model creates a cluster of all the features on the images that it considers to be “normal”, then when you pass a new image to it, it compares the new one with the training cluster and evaluates how similar the two are. If the new image differs a lot from the clusters the anomaly score will be high. Specifically this process is achieved by decomposing the input image into tinier cells which are evaluated separately, and so, in the code, the model is capable of returning a list with all the cells with an anomaly score higher than the threshold.

Both AI models run entirely on the board itself, with no cloud dependency: the decision model is a plain scikit-learn random forest loaded locally, and the vision model runs through the `edge_impulse_linux` runtime — no Docker, no external API calls, and no internet connection required for either one to work.



6. Known limitations & lessons learned

The decision model never sees real sensor data — as explained in section 5.1, its training set is entirely synthetic. This is a deliberate trade-off: the model is only ever as good as the thresholds we hand-coded, and it has never been validated against real, messy, correlated garden data. Still, it gave us a working, probabilistic decision layer instead of a brittle if/else chain.

The vision model detects anomalies, not specific problems. The original plan was a model that could point at a specific issue — a dry or diseased leaf — but that requires a labeled dataset built by hand, region by region, for each defect. We went with unsupervised anomaly detection instead: it only needs photos of a normal, healthy garden to train on, and flags whatever doesn't statistically match, with none of that manual labeling effort. The trade-off is real: in testing, it reliably reacts to a foreign object appearing in frame, but a moderately dry leaf blends into the natural color variation the model already treats as normal, so it won't reliably catch the exact problem we originally wanted it to. On top of that, the webcam's autofocus wasn't always reliable at the working distance we mounted it at, which cost us some of the sharpness the model could have used.

The capacitive soil-moisture probes are the weakest link in the sensor stack. They drift over time and are noisy enough that we average multiple readings and reject outliers in the sketch just to get a usable signal — and even then, the absolute values shouldn't be trusted too far. For a next hardware revision, a more stable and better-calibrated soil sensor is a priority.

The water pumps are consumer-grade and it shows. We hit multiple failures during development with the low-cost pumps we had on hand — not a software problem, but real enough that the actuator code logs and alerts on a failed activation rather than assuming it worked, and it's the first thing we'd upgrade in a physical rebuild.

Battery life is tight for what we're asking of it. Between the board itself, the camera doing periodic inference, and the pump/fans drawing current when active, the current battery setup is undersized for sustained, unattended operation — fine for a demo window, not yet for a real deployment cycle.

7. Conclusions

What started as an idea to make urban food-growing more accessible became, over one summer, a fully working system. From the wood, the sensors, and the wires, to the full operative system, we have built from scratch what we had in mind, we have learned about the value our effort can contribute to the community, and also how to work with high-tech systems.

We're proud of how complete the result is for the time we had, but we're just as proud of understanding exactly where it falls short: the soil-moisture sensors need replacing with something more stable, the decision model needs real garden data instead of synthetic rules to truly earn our trust, and the water pumps need to be swapped for hardware that matches the reliability of everything we built around them. None of that is a surprise to us — it's the roadmap for the next version, and it's one we're excited to keep building.